

JavaScript Arrays — Spread, Destructuring & Advanced Array Patterns

Topics: Array Spread `...`, Array Destructuring, Rest Pattern, Shallow Copy, Combining Arrays, Immutable Updates, Nested References, Function Return Destructuring, Swapping, React State Patterns

1. Spread Operator `...`

JavaScript-এর `...` syntax context অনুযায়ী **Spread** অথবা **Rest** হিসেবে কাজ করতে পারে।

এই document-এ প্রথমে Array-এর **Spread** নিয়ে আলোচনা করব।

Spread-এর basic idea:

একটি array-এর elements-গুলোকে আলাদা আলাদা value হিসেবে ছড়িয়ে দেওয়া।

Example:

```
const numbers = [1, 2, 3];  
  
console.log(...numbers);
```

Output:

```
1 2 3
```

অর্থাৎ:

```
...numbers
```

কে conceptually ভাবতে পারো:

```
1, 2, 3
```

2. Spread দিয়ে Array Copy

একটি array-এর shallow copy তৈরি করার সবচেয়ে common pattern:

```
const original = [1, 2, 3];

const copy = [...original];

console.log(copy);
```

Output:

```
[1, 2, 3]
```

এখন:

```
console.log(original === copy);
```

Output:

```
false
```

কারণ দুইটি আলাদা array।

3. `const` Array Copy করার কারণ

এটা:

```
const numbers = [1, 2, 3];

numbers.push(4);
```

valid।

কারণ `const` variable-কে reassign করা যাবে না, কিন্তু array-এর contents পরিবর্তন করা যায়।

কিন্তু:

```
const numbers = [1, 2, 3];

numbers = [4, 5, 6];
```

এটা error।

Spread দিয়ে নতুন array তৈরি করলে:

```
const numbers = [1, 2, 3];

const newNumbers = [...numbers, 4, 5];
```

এখানে original array পরিবর্তন হচ্ছে না।

4. Spread দিয়ে Array Combine

দুইটি array combine করতে পারো:

```
const frontend = ["HTML", "CSS", "JavaScript"];

const backend = ["Node.js", "Express"];

const skills = [...frontend, ...backend];

console.log(skills);
```

Output:

```
[
  "HTML",
  "CSS",
  "JavaScript",
  "Node.js",
  "Express"
]
```

5. Array-এর শুরুতে Elements Add

```
const numbers = [3, 4, 5];

const result = [1, 2, ...numbers];

console.log(result);
```

Output:

```
[1, 2, 3, 4, 5]
```

6. Array-এর শেষে Elements Add

```
const numbers = [1, 2, 3];

const result = [...numbers, 4, 5];

console.log(result);
```

Output:

```
[1, 2, 3, 4, 5]
```

7. Array-এর মাঝখানে Elements Add

```
const numbers = [1, 2, 5, 6];

const result = [
  ...numbers.slice(0, 2),
  3,
  4,
  ...numbers.slice(2)
];

console.log(result);
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

এটি React state update-এর মতো immutable operations-এ useful।

8. Spread vs Mutation

পুরোনো array mutate করা:

```
const numbers = [1, 2, 3];

numbers.push(4);
```

এখানে original array পরিবর্তন হয়েছে।

Spread ব্যবহার:

```
const numbers = [1, 2, 3];

const newNumbers = [...numbers, 4];
```

এখানে নতুন array তৈরি হয়েছে।

Mental model:

```
push()  
→ existing array modify  
  
[...array, newValue]  
→ new array create
```

9. React-এ Spread খুব Important

React state:

```
const [users, setUsers] = useState([]);
```

নতুন user add:

```
setUsers(prevUsers => [  
  ...prevUsers,  
  newUser  
]);
```

এখানে old array mutate হচ্ছে না।

নতুন array তৈরি হচ্ছে।

10. Array থেকে Element Remove করার Immutable Pattern

Spread-এর সাথে `filter()` combine করা যায়।

```
const numbers = [1, 2, 3, 4, 5];  
  
const result = numbers.filter(number => number !== 3);  
  
console.log(result);
```

Output:

```
[1, 2, 4, 5]
```

React:

```
setUsers(prevUsers =>
  prevUsers.filter(user => user.id !== userId)
);
```

11. Array Update করার Immutable Pattern

ধরো:

```
const users = [
  { id: 1, name: "A", active: false },
  { id: 2, name: "B", active: false }
];
```

ID `2` user-এর `active` true করতে:

```
const updatedUsers = users.map(user =>
  user.id === 2
    ? { ...user, active: true }
    : user
);
```

এখানে দুই ধরনের spread আছে:

```
[...users]
```

array spread

এবং:

```
{ ...user }
```

object spread।

12. Shallow Copy

Spread দিয়ে array copy করলে সেটি **shallow copy**।

Example:

```
const original = [  
  {  
    name: "Shariar"  
  }  
];  
  
const copy = [...original];
```

এখন:

```
console.log(original === copy);
```

Output:

```
false
```

কিন্তু:

```
console.log(original[0] === copy[0]);
```

Output:

```
true
```

এটাই shallow copy-এর মূল বিষয়।

13. কেন Nested Object-এর Reference Same?

ধরো:

```
const original = [  
  {  
    name: "Shariar"  
  }  
];  
  
const copy = [...original];
```

Array নতুন:

```
original → Array A  
copy      → Array B
```

কিন্তু object:

```
Array A → Object X  
Array B → Object X
```

অর্থাৎ দুই array-এর ভিতরের object একই reference।

14. Nested Reference Problem

```
const original = [  
  {  
    name: "Shariar"  
  }  
];  
  
const copy = [...original];  
  
copy[0].name = "Rahim";  
  
console.log(original[0].name);
```

Output:

```
Rahim
```

কারণ:

```
copy[0]
```

এবং:

```
original[0]
```

একই object reference।

15. Nested Object Copy করার Pattern

যদি nested object update করতে হয়:

```
const users = [
  {
    id: 1,
    name: "Shariar",
    profile: {
      city: "Dhaka"
    }
  }
];

const updatedUsers = users.map(user =>
  user.id === 1
    ? {
      ...user,
      profile: {
        ...user.profile,
        city: "Rangpur"
      }
    }
    : user
  );
```

এখানে:

```
...user
```

outer object copy করছে।

এবং:

```
...user.profile
```

nested profile object copy করছে।

16. Spread দিয়ে Array Merge করার সময় Order

Order গুরুত্বপূর্ণ।

```
const first = [1, 2];

const second = [3, 4];

const result = [...first, ...second];

console.log(result);
```

Output:

```
[1, 2, 3, 4]
```

কিন্তু:

```
const result = [...second, ...first];
```

Output:

```
[3, 4, 1, 2]
```

17. Duplicate Values Remove করার Pattern

Spread + `Set`:

```
const numbers = [1, 2, 2, 3, 3, 4];

const uniqueNumbers = [...new Set(numbers)];

console.log(uniqueNumbers);
```

Output:

```
[1, 2, 3, 4]
```

এখানে:

```
new Set(numbers)
```

unique values রাখে।

আর:

```
[...new Set(numbers)]
```

Set-কে আবার array-তে convert করে।

18. Array Destructuring

Array destructuring হলো array থেকে values বের করে আলাদা variables-এ রাখা।

Example:

```
const numbers = [10, 20, 30];

const [a, b, c] = numbers;

console.log(a);
console.log(b);
console.log(c);
```

Output:

```
10
20
30
```

এখানে:

```
const [a, b, c]
```

হলো destructuring pattern।

19. Destructuring Position Based

Array destructuring **position-based**

```
const fruits = ["Apple", "Banana", "Mango"];

const [first, second, third] = fruits;
```

Mapping:

```
first → index 0
second → index 1
third → index 2
```

20. শুধু প্রথম Element নেওয়া

```
const numbers = [10, 20, 30];

const [first] = numbers;

console.log(first);
```

Output:

```
10
```

21. প্রথম দুইটি Element নেওয়া

```
const numbers = [10, 20, 30, 40];

const [first, second] = numbers;

console.log(first);
console.log(second);
```

Output:

```
10
20
```

বাকি values ignore করা হয়েছে।

22. Elements Skip করা

ধরো শুধু first এবং third value দরকার:

```
const numbers = [10, 20, 30];

const [first, , third] = numbers;

console.log(first);
console.log(third);
```

Output:

```
10
30
```

মাঝখানের comma:

```
,
```

দিয়ে second value skip করা হয়েছে।

23. Default Value

যদি array-এর value `undefined` হয়, default value ব্যবহার করা যায়।

```
const numbers = [10];

const [first, second = 20] = numbers;

console.log(first);
console.log(second);
```

Output:

```
10
20
```

24. Default Value কখন কাজ করে?

শুধু `undefined` হলে default value কাজ করে।

```
const values = [undefined];

const [value = 100] = values;

console.log(value);
```

Output:

```
100
```

কিন্তু:

```
const values = [null];

const [value = 100] = values;

console.log(value);
```

Output:

```
null
```

কারণ `null` হলো actual value, `undefined` নয়।

25. Rest Pattern in Destructuring

Array-এর বাকি elements নিতে:

```
const numbers = [1, 2, 3, 4, 5];

const [first, second, ...rest] = numbers;

console.log(first);
console.log(second);
console.log(rest);
```

Output:

```
1
2
[3, 4, 5]
```

এখানে:

```
...rest
```

বাকি elements সংগ্রহ করেছে।

26. Rest Pattern কোথায় থাকতে পারে?

Rest element সাধারণত destructuring pattern-এর **শেষে** থাকতে হয়।

Correct:

```
const [first, ...rest] = numbers;
```

Wrong:

```
const [...rest, last] = numbers;
```

এটা syntax error।

কারণ rest সব remaining values collect করে।

27. Spread vs Rest

দুটোর syntax একই:

...

কিন্তু কাজ context অনুযায়ী আলাদা।

Spread

Values ছড়িয়ে দেয়:

```
const numbers = [1, 2, 3];

const copy = [...numbers];
```

Rest

Values collect করে:

```
const numbers = [1, 2, 3];

const [first, ...rest] = numbers;
```

Mental model:

```
Spread → Expand
Rest    → Collect
```

28. Function Parameters-এ Rest

Rest শুধু array destructuring-এ নয়।

Function-এও ব্যবহার করা যায়:

```
function sum(...numbers) {
  console.log(numbers);
}

sum(10, 20, 30);
```

Output:

```
[10, 20, 30]
```

এখানে `...numbers` সব arguments collect করে array বানিয়েছে।

29. Rest Parameter vs Spread Operator

খুব important difference:

```
function sum(...numbers) {}
```

এখানে:

```
Rest Parameter
```

কারণ multiple arguments collect করছে।

অন্যদিকে:

```
const numbers = [1, 2, 3];

sum(...numbers);
```

এখানে:

```
Spread Operator
```

কারণ array-এর values arguments হিসেবে ছড়িয়ে দিচ্ছে।

30. Spread দিয়ে Function Call

```
const numbers = [10, 20, 30];

console.log(Math.max(...numbers));
```

এটা conceptually:

```
Math.max(10, 20, 30);
```

এর মতো।

31. Array Destructuring দিয়ে Swap

দুইটি variable swap করার elegant way:

```
let a = 10;
let b = 20;

[a, b] = [b, a];

console.log(a);
console.log(b);
```

Output:

```
20
10
```

Traditional temporary variable লাগছে না।

32. Destructuring দিয়ে Function Return

Function যদি array return করে:

```
function getCoordinates() {
  return [100, 200];
}

const [x, y] = getCoordinates();

console.log(x);
console.log(y);
```

Output:

```
100
200
```

33. React `useState()` -এ Destructuring

React-এর:

```
const [count, setCount] = useState(0);
```

এখানে array destructuring হচ্ছে।

Conceptually:

```
useState(0)
  ↓
[value, setter]
  ↓
[count, setCount]
```

React একটি array return করে এবং আমরা destructuring করে দুইটি variable নিচ্ছি।

34. React Hook-এর Destructuring

```
const [name, setName] = useState("");
```

এখানে:

```
name
  → current state

setName
  → state update function
```

Array destructuring-এর practical importance বোঝার জন্য এটি অন্যতম best example।

35. Nested Array Destructuring

Nested array থাকলেও destructure করা যায়।

```
const numbers = [1, [2, 3]];

const [first, [second, third]] = numbers;

console.log(first);
console.log(second);
console.log(third);
```

Output:

```
1
2
3
```

36. Nested Destructuring with Defaults

```
const data = [1, []];

const [first, [second = 20]] = data;

console.log(first);
console.log(second);
```

Output:

```
1
20
```

37. Destructuring দিয়ে First/Last Element

প্রথম element:

```
const numbers = [10, 20, 30, 40];

const [first] = numbers;

console.log(first);
```

শেষ element-এর জন্য সরাসরি destructuring syntax নেই।

তাই:

```
const numbers = [10, 20, 30, 40];

const last = numbers[numbers.length - 1];

console.log(last);
```

অথবা modern JavaScript:

```
const last = numbers.at(-1);
```

38. Spread + Destructuring একসাথে

```
const numbers = [1, 2, 3, 4, 5];

const [first, ...remaining] = numbers;

const newNumbers = [...remaining];

console.log(first);
console.log(newNumbers);
```

Output:

```
1
[2, 3, 4, 5]
```

এখানে প্রথমে rest collect করেছে, তারপর spread দিয়ে নতুন array তৈরি হয়েছে।

39. Array Copy-এর তিনটি Common Way

Spread

```
const copy = [...original];
```

Slice

```
const copy = original.slice();
```

Array.from()

```
const copy = Array.from(original);
```

সবগুলো shallow copy তৈরি করতে পারে।

40. কোন Copy Method ব্যবহার করবে?

Modern JavaScript/React code-এ সাধারণত:

```
const copy = [...array];
```

সবচেয়ে readable এবং common।

`slice()` -ও useful:

```
const copy = array.slice();
```

`Array.from()` useful যখন iterable/array-like source থেকে নতুন array তৈরি করতে চাও।

41. Spread কি Deep Copy করে?

না।

```
const original = [
  {
    name: "A"
  }
];

const copy = [...original];
```

এটা:

Shallow Copy

Deep nested data-এর প্রতিটি nested reference আলাদা করে copy করে না।

42. Deep Copy-এর Basic Idea

যদি nested data সম্পূর্ণ independent copy দরকার হয়, একটি modern built-in option হলো:

```
const copy = structuredClone(original);
```

Example:

```
const original = [
  {
    name: "A",
    address: {
      city: "Dhaka"
    }
  }
];

const copy = structuredClone(original);

copy[0].address.city = "Rangpur";

console.log(original[0].address.city);
```

Output:

তবে `structuredClone()` এবং spread-এর উদ্দেশ্য এক নয়।

সাধারণ React state update-এর ক্ষেত্রে পুরো state deep clone করার বদলে যে level পরিবর্তন হচ্ছে শুধু সেই level-গুলো **immutable**ভাবে **copy** করা বেশি appropriate।

43. Spread দিয়ে React Array Update

Add

```
setItems(prevItems => [  
  ...prevItems,  
  newItem  
]);
```

Remove

```
setItems(prevItems =>  
  prevItems.filter(item => item.id !== id)  
);
```

Update

```
setItems(prevItems =>  
  prevItems.map(item =>  
    item.id === id  
      ? { ...item, completed: true }  
      : item  
  )  
);
```

এগুলো frontend development-এর অত্যন্ত important patterns।

44. Array Reordering with Spread

ধরো:

```
const numbers = [1, 2, 3, 4];
```

শেষের element সামনে আনতে:

```
const result = [  
  numbers[numbers.length - 1],  
  ...numbers.slice(0, -1)  
];  
  
console.log(result);
```

Output:

```
[4, 1, 2, 3]
```

45. Two Arrays-এর Unique Combination

```
const first = [1, 2, 3];  
  
const second = [3, 4, 5];  
  
const unique = [...new Set([...first, ...second])];  
  
console.log(unique);
```

Output:

```
[1, 2, 3, 4, 5]
```

Process:

first + second

↓

[1,2,3,3,4,5]

↓

Set

↓

unique values

↓

spread

↓

array

46. Destructuring-এর Common Mistake

Object এবং Array destructuring এক নয়।

Array:

```
const user = ["Shariar", 31];

const [name, age] = user;
```

Position-based।

Object:

```
const user = {
  name: "Shariar",
  age: 31
};

const { name, age } = user;
```

Property-name-based।

এই দুইটা concept আলাদা করে মনে রাখো।

47. Destructuring Assignment

Declaration-এর সময়:

```
const [a, b] = [10, 20];
```

Existing variables থাকলেও destructuring করা যায়:

```
let a;  
let b;  
  
[a, b] = [10, 20];  
  
console.log(a);  
console.log(b);
```

48. Semicolon-এর Important Edge Case

যদি destructuring assignment statement `[` দিয়ে শুরু হয় এবং আগের statement-এর শেষে semicolon না থাকে, automatic semicolon insertion-এর কারণে unexpected behavior হতে পারে।

যেমন defensiveভাবে:

```
const a = 10;  
  
[a, b] = [20, 30];
```

এখানে আগের statement-এর semicolon রাখা ভালো।

বিশেষ করে code formatting এবং concatenation-এর ক্ষেত্রে এটি মাথায় রাখবে।

49. Spread দিয়ে String Expand করা

Spread শুধু array-এর জন্য নয়।

String iterable হওয়ায়:

```
const name = "Shariar";

const letters = [...name];

console.log(letters);
```

Output:

```
["S", "h", "a", "r", "i", "a", "r"]
```

50. Spread দিয়ে Set থেকে Array

```
const numbers = new Set([1, 2, 3]);

const array = [...numbers];

console.log(array);
```

Output:

```
[1, 2, 3]
```

51. Spread দিয়ে Iterable-এর Concept

Spread সাধারণত iterable values-এর elements expand করতে পারে।

Common iterable examples:

```
Array
String
Set
Map
```

তাই:

```
[..."ABC"]
```

Output:

```
["A", "B", "C"]
```

52. Spread-এর একটি Limitation

এটা:

```
const obj = {  
  name: "Shariar"  
};  
  
const result = [...obj];
```

কাজ করবে না, কারণ সাধারণ object iterable নয়।

Object copy করার জন্য object spread:

```
const result = {  
  ...obj  
};
```

এখানে Array Spread এবং Object Spread-এর context আলাদা।

53. Array Spread vs Object Spread

Array

```
const copy = [...array];
```

Object

```
const copy = { ...object };
```

দুটোতেই মূল idea:

Existing data থেকে নতুন shallow copy তৈরি করা।

54. Advanced Pattern — Remove First Element Without Mutation

Mutation:

```
const numbers = [1, 2, 3, 4];

numbers.shift();
```

Immutable pattern:

```
const numbers = [1, 2, 3, 4];

const result = numbers.slice(1);

console.log(result);
```

Output:

```
[2, 3, 4]
```

55. Advanced Pattern — Remove Last Element Without Mutation

Mutation:

```
numbers.pop();
```

Immutable:

```
const result = numbers.slice(0, -1);
```

অথবা:

```
const result = [...numbers];

result.pop();
```

প্রথমটি সরাসরি নতুন array return করে; দ্বিতীয়টিতে copy করার পর mutation হচ্ছে copy-এর উপর।

56. Advanced Pattern — Insert Without Mutation

```
const numbers = [1, 2, 5, 6];

const index = 2;

const result = [
  ...numbers.slice(0, index),
  3,
  4,
  ...numbers.slice(index)
];

console.log(result);
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

57. Advanced Pattern — Replace Without Mutation

```
const numbers = [10, 20, 30, 40];

const index = 2;

const result = [
  ...numbers.slice(0, index),
  99,
  ...numbers.slice(index + 1)
];

console.log(result);
```

Output:

```
[10, 20, 99, 40]
```

58. Advanced Pattern — Swap Array Elements

```
const numbers = [10, 20, 30, 40];

[numbers[0], numbers[2]] = [numbers[2], numbers[0]];

console.log(numbers);
```

Output:

```
[30, 20, 10, 40]
```

এখানে array mutate হয়েছে।

যদি React state-এর immutable version দরকার হয়:

```
setNumbers(prev => {  
  const next = [...prev];  
  
  [next[0], next[2]] = [next[2], next[0]];  
  
  return next;  
});
```

59. Practical Example — Todo List

```
const todos = [  
  { id: 1, title: "Learn JavaScript", completed: false },  
  { id: 2, title: "Practice React", completed: false }  
];
```

Add Todo

```
const newTodo = {  
  id: 3,  
  title: "Practice Next.js",  
  completed: false  
};  
  
const updatedTodos = [  
  ...todos,  
  newTodo  
];
```

Complete Todo

```
const updatedTodos = todos.map(todo =>  
  todo.id === 2  
    ? { ...todo, completed: true }  
    : todo  
);
```

Delete Todo

```
const updatedTodos = todos.filter(todo => todo.id !== 1);
```

এগুলোই React state management-এর foundation-এর অংশ।

60. Practical Example — API Response

ধরো API থেকে:

```
const response = [  
  ["Shariar", 31],  
  ["Rahim", 25],  
  ["Karim", 28]  
];
```

প্রথম user-এর data:

```
const [firstUser] = response;  
  
console.log(firstUser);
```

Output:

```
["Shariar", 31]
```

তারপর:

```
const [name, age] = firstUser;  
  
console.log(name);  
console.log(age);
```

Output:

```
Shariar  
31
```

61. Practical Example — Function Arguments

```
function calculateTotal(a, b, c) {  
  return a + b + c;  
}  
  
const prices = [100, 200, 300];  
  
const total = calculateTotal(...prices);  
  
console.log(total);
```

Output:

```
600
```

Spread array-এর values-কে function arguments হিসেবে expand করেছে।

62. Common Mistakes

Mistake 1 — Spread Deep Copy করে ভাবা

Wrong assumption:

```
const copy = [...original];
```

মানে সব nested data completely independent।

না।

এটা shallow copy।

Mistake 2 — Spread নিজে Mutation Prevent করে

Spread:

```
const copy = [...original];
```

নতুন array বানায়।

কিন্তু:

```
copy.push(10);
```

copy mutate করতে পারে।

Spread immutable data structure বানানোর magic lock নয়।

Mistake 3 — Rest এবং Spread গুলিয়ে ফেলা

```
const copy = [...numbers];
```

→ Spread

```
const [first, ...rest] = numbers;
```

→ Rest

Mistake 4 — Rest মাঝখানে দেওয়া

Wrong:

```
const [first, ...rest, last] = numbers;
```

Rest element শেষেই থাকতে হবে।

Mistake 5 — Destructuring-এ Order ভুলে যাওয়া

```
const numbers = [10, 20, 30];
```

```
const [a, b] = numbers;
```

এখানে:

```
a = 10  
b = 20
```

Property name দেখে নয়, position দেখে assign হয়েছে।

63. Output Prediction Practice

Question 1

```
const numbers = [1, 2, 3];  
  
const copy = [...numbers];  
  
console.log(copy === numbers);
```

Output কী?

Question 2

```
const numbers = [10, 20, 30];  
  
const [first, ...rest] = numbers;  
  
console.log(first);  
console.log(rest);
```

Question 3

```
const a = [1, 2];  
  
const b = [3, 4];  
  
const result = [...a, ...b];  
  
console.log(result);
```

Question 4

```
const numbers = [10, 20, 30];

const [first, , third] = numbers;

console.log(first);
console.log(third);
```

Question 5

```
const numbers = [1, 2, 3];

const [a, b = 100, c = 200] = numbers;

console.log(a, b, c);
```

Question 6

```
const numbers = [1];

const [a, b = 20, c = 30] = numbers;

console.log(a, b, c);
```

Question 7

```
const original = [
  {
    name: "A"
  }
];

const copy = [...original];

console.log(original === copy);
console.log(original[0] === copy[0]);
```

দুইটা output কেন আলাদা হবে?

64. Coding Practice

Challenge 1 — Combine Arrays

```
const frontend = ["HTML", "CSS", "JavaScript"];

const backend = ["Node", "Express"];
```

Spread ব্যবহার করে একটি `skills` array বানাও।

Expected:

```
[
  "HTML",
  "CSS",
  "JavaScript",
  "Node",
  "Express"
]
```

Challenge 2 — First and Rest

```
const numbers = [10, 20, 30, 40, 50];
```

Destructuring দিয়ে:

```
first = 10
rest = [20, 30, 40, 50]
```

বের করো।

Challenge 3 — Swap

```
let a = 100;
let b = 200;
```

Temporary variable ছাড়া swap করো।

Expected:

```
a = 200
b = 100
```

Challenge 4 — Immutable Add

```
const users = [
  { id: 1, name: "A" },
  { id: 2, name: "B" }
];
```

Spread ব্যবহার করে নতুন user add করো।

Challenge 5 — Immutable Update

```
const users = [
  { id: 1, name: "A", active: false },
  { id: 2, name: "B", active: false }
];
```

ID `2` user-এর `active` true করো।

Original array mutate করা যাবে না।

Challenge 6 — Remove Duplicate

```
const numbers = [1, 2, 2, 3, 3, 4, 4];
```

Spread + `Set` ব্যবহার করে:

```
[1, 2, 3, 4]
```

বানাও।

Challenge 7 — Function Arguments

```
function add(a, b, c) {  
  return a + b + c;  
}  
  
const numbers = [10, 20, 30];
```

Spread ব্যবহার করে function call করো।

Expected:

```
60
```

65. Interview Questions

Beginner

1. Spread operator কী?
2. Array copy করতে spread কীভাবে ব্যবহার করবে?
3. Array combine করতে spread কীভাবে ব্যবহার করবে?
4. Array destructuring কী?
5. Array destructuring position-based কেন?
6. Rest pattern কী?
7. Spread এবং Rest-এর difference কী?
8. Array destructuring-এ default value কীভাবে দেওয়া হয়?
9. Destructuring দিয়ে variable swap কীভাবে করা যায়?

Intermediate

10. Spread কি deep copy তৈরি করে?
11. Shallow copy কী?

12. Nested object-এর reference কেন same থাকে?
 13. `const` array এবং immutability-এর সম্পর্ক কী?
 14. React state update-এ spread কেন ব্যবহার করা হয়?
 15. `map()` + object spread দিয়ে object update কীভাবে করা যায়?
 16. `filter()` দিয়ে immutable deletion কীভাবে করা যায়?
 17. Spread দিয়ে function arguments কীভাবে পাঠানো যায়?
 18. Rest parameter এবং spread operator-এর difference কী?
 19. Array destructuring এবং object destructuring-এর difference কী?
 20. `structuredClone()` এবং spread-এর difference কী?
-

66. সবচেয়ে গুরুত্বপূর্ণ Mental Model

Spread

```
...array
  ↓
Expand / ছড়িয়ে দাও
```

Example:

```
const result = [...numbers];
```

Rest

```
...rest
  ↓
Collect / সংগ্রহ করো
```

Example:

```
const [first, ...rest] = numbers;
```

Destructuring

```
Array
  ↓
Position অনুযায়ী
  ↓
Variables
```

Example:

```
const [first, second] = numbers;
```

Shallow Copy

```
New Array
  ↓
Same nested references
```

React Immutable Update

```
Old State
  ↓
Create New Array
  ↓
Change Required Item
  ↓
Return New State
```

67. Quick Revision

`[...array]`

→ array spread

→ shallow copy

`[...a, ...b]`

→ combine arrays

`[1, 2, ...array]`

→ beginning-এ add

`[...array, 4, 5]`

→ ending-এ add

`const [a, b] = array`

→ array destructuring

`const [first, , third] = array`

→ skip element

`const [first, second = 10] = array`

→ default value

`const [first, ...rest] = array`

→ rest pattern

`function sum(...numbers)`

→ rest parameter

`sum(...numbers)`

→ spread arguments

`[a, b] = [b, a]`

→ swap

`[...array]`

→ shallow copy

`structuredClone(array)`

→ deep structured clone

```
{ ...object }  
→ object shallow copy
```

68. Array Section — Final Mental Map

এখন পুরো **Section 6 — Arrays** একসাথে দেখলে:

ARRAY

- |
 - |— Fundamentals
 - | |— Creation
 - | |— Indexing
 - | |— length
 - | |— Nested arrays
 - | |— Array.isArray()
 - | |— Array.from()
 - | |— Array.of()
- |
 - |— Mutation & Copying
 - | |— push()
 - | |— pop()
 - | |— shift()
 - | |— unshift()
 - | |— splice()
 - | |— slice()
 - | |— Mutable vs Immutable
 - | |— Shallow Copy
- |
 - |— Iteration & Transformation
 - | |— map()
 - | |— filter()
 - | |— reduce()
 - | |— forEach()
 - | |— find()
 - | |— findIndex()
 - | |— some()
 - | |— every()
 - | |— includes()
 - | |— indexOf()
 - | |— flat()
 - | |— flatMap()
- |
 - |— Advanced Patterns
 - | |— Spread (...)
 - | |— Destructuring
 - | |— Rest Pattern
 - | |— Array Copying
 - | |— Combining Arrays
 - | |— Swapping

69. Array-এর সবচেয়ে Important Rules

1. Array index starts at `0`.
2. `length` হলো *property*, *method* নয়।
3. `push/pop/shift/unshift/splice` মূল array mutate করতে পারে।
4. `slice()` নতুন array return করে এবং original array mutate করে না।
5. `map()` → *transform*.
6. `filter()` → *select*.
7. `reduce()` → *accumulate*.
8. `find()` → *first matching element*.
9. `some()` → *at least one*.
10. `every()` → *all*.
11. `...array` → *spread/expand*.
12. `[first, ...rest]` → *rest/collect*.
13. Array destructuring position-based.
14. Spread copy হলো *shallow copy*.
15. React state-এর array সরাসরি mutate না করে নতুন array তৈরি করা উচিত।
16. Nested object update-এর সময় প্রয়োজনীয় nested level-ও copy করতে হয়।

70. Final Array Mastery Checklist

এই section শেষ করার পর তোমার এগুলো confidently করতে পারা উচিত:

- ☐ Array create করতে পারা
- ☐ Index দিয়ে value access/update করতে পারা
- ☐ Nested array বুঝতে পারা
- ☐ `Array.isArray()` ব্যবহার করতে পারা
- ☐ `Array.from()` এবং `Array.of()` বুঝতে পারা
- ☐ `push/pop/shift/unshift` ব্যবহার করতে পারা
- ☐ `splice()` এবং `slice()` difference explain করতে পারা
- ☐ Mutable বনাম immutable বুঝতে পারা
- ☐ `map()` দিয়ে transformation করতে পারা
- ☐ `filter()` দিয়ে selection করতে পারা
- ☐ `reduce()` দিয়ে accumulation করতে পারা
- ☐ `forEach()` কখন ব্যবহার করতে হবে বুঝতে পারা
- ☐ `find()` বনাম `filter()` explain করতে পারা
- ☐ `some()` বনাম `every()` explain করতে পারা
- ☐ `includes()` এবং `indexOf()` ব্যবহার করতে পারা
- ☐ `flat()` এবং `flatMap()` বুঝতে পারা
- ☐ Spread দিয়ে array copy/combine করতে পারা
- ☐ Array destructuring করতে পারা
- ☐ Rest pattern ব্যবহার করতে পারা
- ☐ Spread বনাম Rest explain করতে পারা
- ☐ Shallow copy এবং nested reference বুঝতে পারা
- ☐ React state-এর array immutableভাবে update করতে পারা

Final Takeaway

JavaScript Array-এর ক্ষেত্রে syntax মুখস্থ করার চেয়ে **problem** → **correct method** mapping সবচেয়ে গুরুত্বপূর্ণ।

Change every item

↓
`map()`

Keep some items

↓
`filter()`

Create one result

↓
`reduce()`

Do something for each item

↓
`forEach()`

Find first item

↓
`find()`

Find first index

↓
`findIndex()`

At least one?

↓
`some()`

Everyone?

↓
`every()`

Does value exist?

↓
`includes()`

Need value's index?

↓
`indexOf()`

Flatten?

↓

`flat()`

Map + flatten?

↓

`flatMap()`

Copy/combine/expand?

↓

Spread

Extract values?

↓

Destructuring

Collect remaining values?

↓

Rest

এখন Section 6 — Arrays পুরোপুরি complete. 🚀